实验1：多层神经网络的 numpy 实现

姓名： 张合　　**学号：** M202573073　　**日期：** 2026/03/25

一、实验目的

1. 掌握全连接神经网络前向传播与反向传播的推导与实现
2. 理解激活函数、损失函数、正则化方法的作用与区别
3. 通过纯 numpy 手动实现神经网络，深入理解深度学习框架底层机制
4. 探索超参数（学习率、网络结构、激活函数等）对模型性能的影响规律

二、实验原理

2.1 网络结构

本实验采用全连接多层感知机（MLP），基准结构如下：

```
Input(784) → FC(256) → ReLU → Dropout → FC(128) → ReLU → Dropout → FC(10) → Softmax
```

输入为 MNIST 图像展平后的 784 维向量，输出为 10 个类别的概率分布。

2.2 前向传播

第 l 层的计算过程：

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}, \quad A^{(l)} = f!\left(Z^{(l)}\right)$$

其中 $f(\cdot)$ 为激活函数，输出层使用 Softmax：

$$\hat{y}_k = \mathrm{Softmax}(Z^{(L)})_k = \frac{e^{Z_k^{(L)}}}{\sum_j e^{Z_j^{(L)}}}$$

2.3 损失函数

交叉熵损失（含 L2 正则化）：

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log\hat{y}_{ik} + \frac{\lambda}{2}\sum_{l}|W^{(l)}|_F^2$$

```python
def cross_entropy_loss(probs, Y):
    n = len(Y)
    return -np.sum(Y * np.log(probs + 1e-9)) / n
```

2.4 反向传播

Softmax + 交叉熵联合求导化简后，输出层梯度为：

$$\frac{\partial \mathcal{L}}{\partial Z^{(L)}} = \frac{1}{N}(\hat{Y} - Y)$$

隐藏层依链式法则逐层向前传递：

$$\frac{\partial \mathcal{L}}{\partial Z^{(l)}} = \frac{\partial \mathcal{L}}{\partial A^{(l)}} \odot f'!\left(Z^{(l)}\right), \qquad \frac{\partial \mathcal{L}}{\partial W^{(l)}} = \left(A^{(l-1)}\right)^{\top} \frac{\partial \mathcal{L}}{\partial Z^{(l)}} + \lambda W^{(l)}$$

```python
def backward(self, dA, l2_lambda=0.0):
        """
        dA: 上游梯度（已含 /N 归一化）
        返回 dX 传递给上一层
        """
        dZ = dA * self.act_grad(self._Z)
        self.dW = self._X.T @ dZ + l2_lambda * self.W   # L2 正则梯度
        self.db = dZ.sum(axis=0, keepdims=True)
        return dZ @ self.W.T
```

2.5 激活函数

激活函数	表达式	导数	特点
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$	正区间梯度恒为 1，缓解梯度消失
Leaky ReLU	$\max(\alpha z, z)$	1 或 α	负区间保留小梯度，避免神经元死亡
Tanh	$\tanh(z)$	$1 - \tanh^2(z)$	输出零中心，饱和区仍有梯度消失风险
Sigmoid	$\frac{1}{1+e^{-z}}$	$s(1-s)$	输出非零中心，深层网络易梯度消失

2.6 优化器

本实验实现了三种优化器：

优化器	更新公式	特点
SGD	$W \leftarrow W - \eta \nabla W$	简单，收敛速度依赖学习率选取
Momentum	$v \leftarrow \beta v + \eta \nabla W$；$W \leftarrow W - v$	积累历史梯度，加速收敛、减小震荡
Adam	一阶矩 + 二阶矩 + 偏差修正	自适应学习率，收敛快且对初始值不敏感

Adam 更新规则（$\beta_1 = 0.9$，$\beta_2 = 0.999$，$\epsilon = 10^{-8}$）：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla W, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla W)^2, \quad W \leftarrow W - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

2.7 进阶优化技术

- **Dropout（Inverted Dropout）**：训练时以概率 $1-p$ 随机置零激活值并等比缩放，推理时直接输出，相当于对指数级子网络的隐式集成
- **He 初始化**（适合 ReLU 系列）：$W \sim \mathcal{N}(0,\ \sqrt{2/n_{\text{in}}})$，保持各层激活值方差稳定
- **Xavier 初始化**（适合 Tanh/Sigmoid）：$W \sim \mathcal{N}(0,\ \sqrt{1/n_{\text{in}}})$，兼顾前向与反向传播的方差一致性

三、实验步骤

3.1 数据预处理

使用 `sklearn.datasets.fetch_openml` 加载 MNIST 数据集（70,000 张 28×28 灰度图像），处理流程：

1. 像素值归一化至 $[0,1]$：`X /= 255.0`
2. 标签 One-hot 编码为 10 维向量
3. 随机打乱后按 **8:2** 划分，得到训练集 56,000 条、验证集 14,000 条

3.2 模块化设计

代码分为三个模块：

文件	主要内容
`utils.py`	激活函数及其导数、Softmax、交叉熵损失、He/Xavier 初始化、数据加载、GPU/CPU 自动切换
`model.py`	`FullyConnectedLayer`（前向/反向）、`DropoutLayer`（Inverted Dropout）、`MLP`（含 SGD/Momentum/Adam）
`train.py`	Mini-batch 训练循环、梯度数值检验、三组超参数对比实验、损失曲线/混淆矩阵/错误样本可视化

3.3 梯度数值检验

在正式训练前，用中心差分公式对随机选取的参数做数值梯度验证：

$$\nabla_{\text{num}} W_{ij} = \frac{\mathcal{L}(W_{ij}+\varepsilon) - \mathcal{L}(W_{ij}-\varepsilon)}{2\varepsilon}, \qquad e = \frac{|\nabla_{\text{num}} - \nabla_{\text{ana}}|}{|\nabla_{\text{num}}| + |\nabla_{\text{ana}}| + 10^{-10}}$$

检验结构为 784→16(ReLU)→10(Softmax)，$\varepsilon = 10^{-5}$，对 W1、b1、W2、b2 各随机抽取参数验证。

> 实验结果：最大相对误差为 $\mathbf{1.56 \times 10^{-7}}$，远低于阈值 10^{-4}，**反向传播验证通过。**

3.4 超参数对比实验

设计三组对比实验，每组训练 30 个 epoch：

实验组	隐藏层结构	激活函数	学习率	Batch Size	L2 系数	Dropout 保留率	优化器
Baseline	[256, 128]	ReLU	0.01	64	1e-4	0.8	Adam
Group 1	[512, 256]	Leaky ReLU	0.005	128	1e-3	0.8	Adam
Group 2	[128]	Tanh	0.02	32	0	无	Momentum

四、实验结果

4.1 训练过程

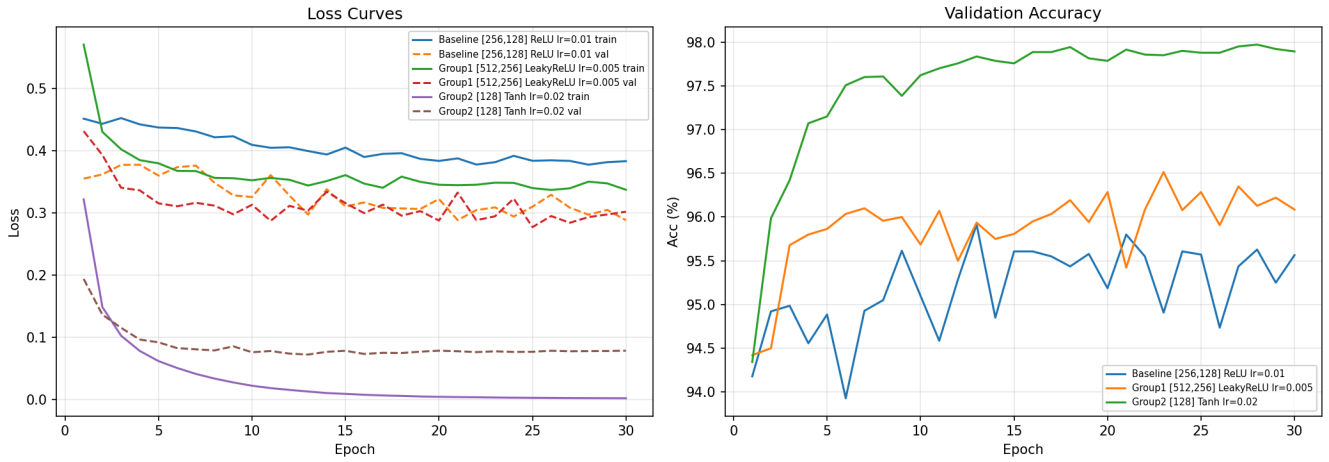

图1：三组实验的训练/验证损失曲线（左）与验证准确率曲线（右）

各组实验关键 epoch 的指标如下：

实验组	Epoch 1	Epoch 5	Epoch 10	Epoch 20	Epoch 30	最终训练损失	最终验证损失
Baseline [256,128] ReLU	94.18%	94.89%	95.10%	95.19%	**95.56%**	0.3831	0.2882
Group 1 [512,256] Leaky ReLU	94.42%	95.86%	95.69%	96.29%	**96.09%**	0.3371	0.3019
Group 2 [128] Tanh	94.34%	97.15%	97.62%	97.79%	**97.89%**	0.0020	0.0784

三组实验均超过 95% 的准确率要求，其中 **Group 2（[128] Tanh + Momentum）表现最佳，达到 97.89%。**

从损失曲线可以观察到：

- Baseline 与 Group 1 训练/验证损失差距较小，L2 + Dropout 有效抑制了过拟合
- Group 2 训练损失迅速降至接近 0（最终 0.0020），但验证损失稳定在 0.0784，两者差距约 39 倍，存在明显

过拟合

- Group 2 的验证准确率曲线最为平滑，第 5 epoch 即达 97.15%，收敛最快
- Group 1 的验证准确率曲线震荡最大，与较大的 batch size（128）和较低的学习率（0.005）的组合有关

4.2 混淆矩阵分析

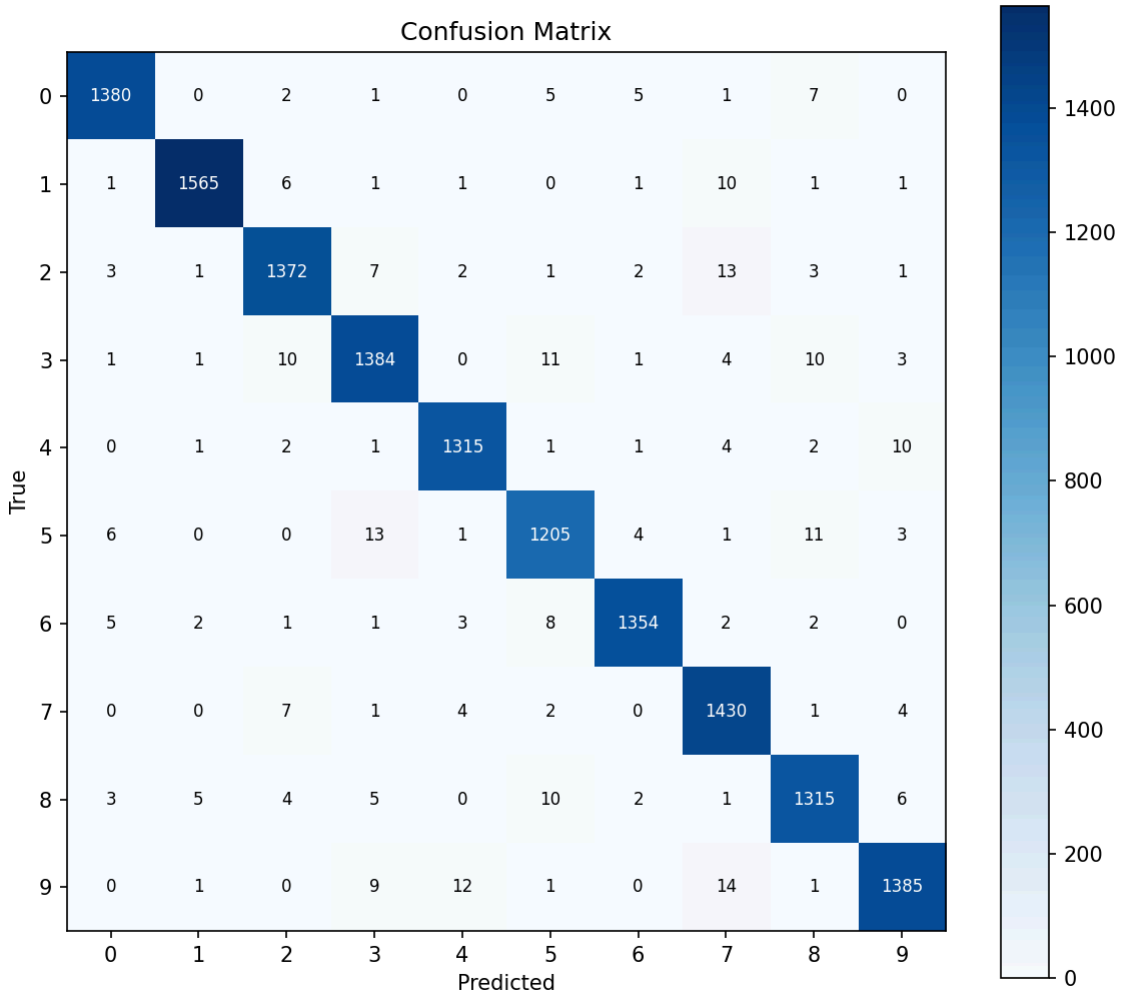

图2：最佳模型（Group 2）在验证集（14,000 条）上的混淆矩阵

各类别识别准确率（由混淆矩阵对角线计算）：

数字	0	1	2	3	4	5	6	7	8	9
正确 / 总数	1380/1401	1565/1587	1372/1405	1384/1425	1315/1337	1205/1244	1354/1378	1430/1449	1315/1351	1385/1423
准确率	98.50%	98.61%	97.65%	97.12%	98.35%	96.86%	98.26%	98.69%	97.34%	97.33%

主要混淆规律：

- **5 → 3**（13 例）：最显著的混淆对，两者上半部分笔画形态高度相似
- **2 → 7**（13 例）、**9 → 7**（14 例）：均与"7"的竖直长笔划混淆
- **9 → 4**（12 例）：顶部是否封口在部分书写风格下难以区分
- 数字 **5**（96.86%）和 **9**（97.33%）的准确率相对最低，是模型的薄弱类别

4.3 错误样本分析

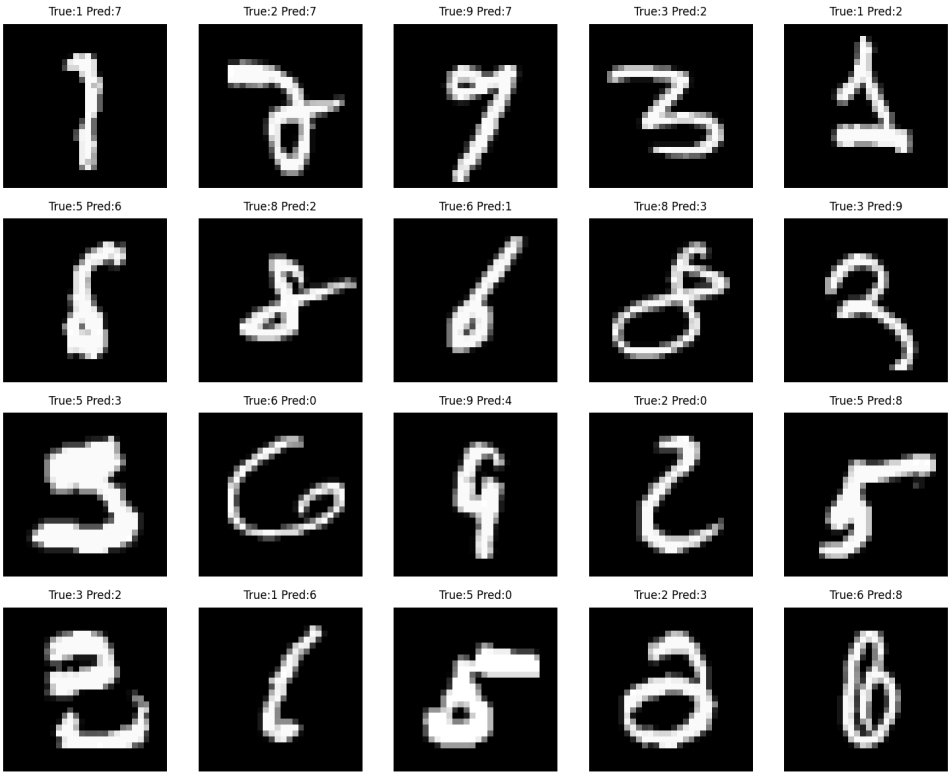

图3：验证集上 20 个典型错误样本（标题格式：真实标签 / 预测标签）

从错误样本可归纳出三类主要错误模式：

书写变形：如 True:1 Pred:7，"1"写成了带斜划的形态；True:2 Pred:7 中"2"底部弯钩消失，仅剩竖划，即便人眼也难以判断。

形态相近：如 True:5 Pred:6、True:8 Pred:2，数字整体轮廓与预测类别高度相似，属于模型合理的边界误判。

笔画不完整：如 True:6 Pred:1 中"6"的圆弧部分缺失，True:5 Pred:0 中"5"顶部横划被截断形似"0"。

这些错误案例表明，模型的失败主要源于书写质量问题，而非系统性分类偏差。

五、结果分析与讨论

5.1 不同激活函数对梯度消失的影响

Sigmoid 和 Tanh 在输入绝对值较大时导数趋近 0，多层叠加后梯度指数衰减，即**梯度消失**。ReLU 在正区间导数恒为 1，完全截断了衰减路径；但输入持续为负时输出恒零（"死亡 ReLU"），Leaky ReLU 通过保留斜率 $\alpha = 0.01$ 缓解这一问题。

然而实验结果呈现与直觉相悖的现象：**单隐层 Tanh（Group 2）最终准确率 97.89%，高于两层 ReLU（Baseline，95.56%）**。原因有三：第一，MNIST 属于浅层任务，单隐层已足够提取特征，梯度消失威胁有限；第二，Tanh 输出零中心，有利于后一层的梯度传播，在浅层结构中优势更明显；第三，Group 2 无正则化约束，模型容量被充分利用，训练损失降至 0.002，但代价是明显的过拟合。

5.2 正则化方法如何平衡偏差-方差

正则化手段	作用机制	对偏差的影响	对方差的影响	本实验表现
L2 正则（λ=1e-4）	惩罚大权重，限制模型复杂度	略微增大	显著降低	Baseline 训练/验证损失差距约 0.09
Dropout（保留率 0.8）	随机丢弃神经元，隐式集成	略微增大	显著降低	与 L2 配合，过拟合控制良好
无正则化（Group 2）	—	偏差极小	方差大	训练损失 0.002，验证损失 0.078，差距 39 倍

从损失曲线可以定量地看出偏差-方差权衡：Group 2 训练损失最低（偏差最小），但训练/验证损失差距最大（方差最大），是典型的过拟合形态；Baseline 和 Group 1 的差距均在 0.05~0.10 以内，正则化效果明显。值得注意的是，尽管 Group 2 过拟合严重，其验证准确率仍最高，说明在数据量充足（训练集 56,000 条）时，即使过拟合，模型已学到足够鲁棒的特征表示。

5.3 网络深度与模型性能的关系

模型	隐藏层	参数量（估算）	最终验证准确率	训练损失	验证损失
Group 2	[128]	≈ 101K	**97.89%**	0.0020	0.0784
Baseline	[256, 128]	≈ 234K	95.56%	0.3831	0.2882
Group 1	[512, 256]	≈ 534K	96.09%	0.3371	0.3019

参数量更多并不必然带来更高准确率。 Group 2 参数最少却准确率最高；Group 1 参数量是 Group 2 的 5 倍，准确率反而低 1.8 个百分点。根本原因在于：MNIST 的特征维度不高，浅层网络已具备足够的表达能力；更深更宽的网络叠加了 Dropout 和 L2 正则后，有效容量被压缩，反而限制了其发挥。这一现象提示，**网络结构的选择应与任务难度和正则化策略协同考虑**，盲目增加深度/宽度而不调整其他超参数，收益有限甚至适得其反。

5.4 学习率与优化器对收敛的影响

从验证准确率曲线可以观察到三种不同的收敛特征：

- **Baseline（Adam，lr=0.01，batch=64）**：曲线震荡较大，早期准确率波动明显，说明学习率偏高；Adam 的自适应机制虽然缓解了震荡但未完全消除
- **Group 1（Adam，lr=0.005，batch=128）**：收敛节奏较稳定，但 epoch 15 后仍有波动；较大的 batch 使梯度估计更准确，但整体收敛速度相对偏慢
- **Group 2（Momentum，lr=0.02，batch=32）**：曲线最为平滑，收敛最快——第 5 epoch 即达 97.15%，第 10 epoch 达 97.62%，此后趋于稳定

Adam 的自适应性在本实验中并未显现出相对 Momentum 的优势，原因可能在于任务较简单、损失面较平滑，Momentum + 合适学习率已足够找到全局最优区域。小 batch size（32）引入的梯度噪声在一定程度上也起到了隐式正则的作用，帮助模型跳出局部极小值。

六、结论

本实验基于纯 numpy 手动实现了完整的多层感知机，涵盖前向传播、反向传播、多种激活函数（ReLU/Leaky ReLU/Tanh/Sigmoid）、交叉熵损失、L2 正则化、Inverted Dropout、Mini-batch 梯度下降以及 SGD/Momentum/Adam 三种优化器，并采用 He/Xavier 参数初始化。梯度数值检验最大相对误差为 1.56×10^{-7}，验证了反向传播实现的正确性。

三组超参数实验均达到 95% 以上的验证准确率，**最佳模型为 Group 2（[128] Tanh + Momentum，无正则化），验证集准确率达到 97.89%**，对应混淆矩阵显示各类别准确率均在 96.86% 以上，满足实验要求。

主要结论：

1. **激活函数**：ReLU 在深层网络中更能抵抗梯度消失；在浅层任务上，Tanh 的零中心输出特性使其收敛更快、最终精度更高
2. **正则化**：L2 + Dropout 的组合有效控制过拟合；数据量充足时，即使过拟合，模型仍可达到较高验证精度
3. **网络规模**：对 MNIST 而言单隐层（128 节点）已足够，盲目增加深度/宽度在未调整正则化策略时收益有限
4. **优化器与学习率**：Momentum + 小 batch 在本任务上优于 Adam；合理的学习率和 batch size 的组合比优化器本身的选择更为关键

七、附实验数据图

```
═══ 梯度数值检验 ═══

── 梯度数值检验 ──
  最大相对误差: 1.56e-07  ✓ 通过
正在加载 MNIST（首次需联网，约需1分钟）...
训练集: (56000, 784)  验证集: (14000, 784)  设备: CPU

═══ 实验: Baseline [256,128] ReLU lr=0.01 ═══
  Epoch   1/30 | train_loss=0.4514 | val_loss=0.3550 | val_acc=94.18%
  Epoch   5/30 | train_loss=0.4372 | val_loss=0.3599 | val_acc=94.89%
  Epoch  10/30 | train_loss=0.4093 | val_loss=0.3254 | val_acc=95.10%
  Epoch  15/30 | train_loss=0.4049 | val_loss=0.3102 | val_acc=95.61%
  Epoch  20/30 | train_loss=0.3835 | val_loss=0.3222 | val_acc=95.19%
  Epoch  25/30 | train_loss=0.3837 | val_loss=0.3100 | val_acc=95.57%
  Epoch  30/30 | train_loss=0.3831 | val_loss=0.2882 | val_acc=95.56%
  最终验证准确率: 95.56%

═══ 实验: Group1 [512,256] LeakyReLU lr=0.005 ═══
═══ 实验: Group1 [512,256] LeakyReLU lr=0.005 ═══
  Epoch   1/30 | train_loss=0.5705 | val_loss=0.4314 | val_acc=94.42%
  Epoch   1/30 | train_loss=0.5705 | val_loss=0.4314 | val_acc=94.42%
  Epoch   5/30 | train_loss=0.3797 | val_loss=0.3153 | val_acc=95.86%
  Epoch   5/30 | train_loss=0.3797 | val_loss=0.3153 | val_acc=95.86%
  Epoch  10/30 | train_loss=0.3524 | val_loss=0.3130 | val_acc=95.69%
  Epoch  15/30 | train_loss=0.3607 | val_loss=0.3156 | val_acc=95.81%
  Epoch  10/30 | train_loss=0.3524 | val_loss=0.3130 | val_acc=95.69%
  Epoch  15/30 | train_loss=0.3607 | val_loss=0.3156 | val_acc=95.81%
  Epoch  15/30 | train_loss=0.3607 | val_loss=0.3156 | val_acc=95.81%
  Epoch  20/30 | train_loss=0.3452 | val_loss=0.2877 | val_acc=96.29%
  Epoch  20/30 | train_loss=0.3452 | val_loss=0.2877 | val_acc=96.29%
  Epoch  25/30 | train_loss=0.3398 | val_loss=0.2770 | val_acc=96.29%
  Epoch  30/30 | train_loss=0.3371 | val_loss=0.3019 | val_acc=96.09%
  Epoch  25/30 | train_loss=0.3398 | val_loss=0.2770 | val_acc=96.29%
  Epoch  30/30 | train_loss=0.3371 | val_loss=0.3019 | val_acc=96.09%
  最终验证准确率: 96.09%


═══ 实验: Group2 [128] Tanh lr=0.02 ═══
  Epoch   1/30 | train_loss=0.3217 | val_loss=0.1938 | val_acc=94.34%
  Epoch   5/30 | train_loss=0.0617 | val_loss=0.0919 | val_acc=97.15%
  Epoch  10/30 | train_loss=0.0221 | val_loss=0.0759 | val_acc=97.62%
  Epoch   5/30 | train_loss=0.0617 | val_loss=0.0919 | val_acc=97.15%
  Epoch  10/30 | train_loss=0.0221 | val_loss=0.0759 | val_acc=97.62%
  Epoch  10/30 | train_loss=0.0221 | val_loss=0.0759 | val_acc=97.62%
  Epoch  15/30 | train_loss=0.0090 | val_loss=0.0783 | val_acc=97.76%
  Epoch  15/30 | train_loss=0.0090 | val_loss=0.0783 | val_acc=97.76%
  Epoch  20/30 | train_loss=0.0043 | val_loss=0.0784 | val_acc=97.79%
  Epoch  25/30 | train_loss=0.0027 | val_loss=0.0767 | val_acc=97.88%
  Epoch  30/30 | train_loss=0.0020 | val_loss=0.0784 | val_acc=97.89%
  最终验证准确率: 97.89%
```